Cache Control

1.0

Создано системой Doxygen 1.9.8

1 CacheControl — система управления страничным кэшем 1
1.1 Содержание . . . 1
1.2 Возможности . . . 1
1.2.1 Требования . . . 2
1.3 Конфигурация . . . 2
1.3.1 Формат конфигурационного файла . . . 2
1.3.2 Поддерживаемые политики . . . 2
1.3.3 Параметры . . . 2
1.3.4 Пример конфигурационного файла . . . 2
1.4 Уровни логирования . . . 3
1.5 Использование . . . 3
1.5.1 Запуск демона . . . 3
1.5.2 Запуск медиатора . . . 3
1.6 Команды медиатора . . . 3
1.7 Сигналы . . . 3
1.8 Структура проекта . . . 4
1.8.1 Модули демона (src/cache_control/) . . . 4
1.8.2 Модули медиатора (src/cache_control_mediator/) . . . 5
1.9 Сборка . . . 5
1.9.1 Доступные цели Make . . . 5
1.9.2 Сборка проекта . . . 5
1.9.3 Генерация документации . . . 5
1.10 Документация . . . 6

2 Алфавитный указатель структур данных 7
2.1 Структуры данных . . . 7

3 Список файлов 9
3.1 Файлы . . . 9

4 Структуры данных 11
4.1 Структура cache_control . . . 11
4.2 Структура command_t . . . 11
4.3 Структура file . . . 12
4.4 Структура file_info . . . 12
4.5 Структура file_monitoring . . . 12
4.6 Структура file_stat . . . 13
4.7 Структура list . . . 13
4.8 Структура mediator_command . . . 13
4.9 Структура mqfd_t . . . 13
4.10 Структура node . . . 14
4.11 Структура replay_t . . . 14
4.12 Структура reply_t . . . 14

5 Файлы 15

5.1 cache_control.h . . . 15
5.2 command_listener.h . . . 15
5.3 context.h . . . 16
5.4 daemonize.h . . . 16
5.5 events_stuff.h . . . 16
5.6 file_loading.h . . . 16
5.7 file_monitoring.h . . . 17
5.8 file_statistics.h . . . 17
5.9 inotify_stuff.h . . . 18
5.10 options.h . . . 18
5.11 parse_config.h . . . 18
5.12 signal_handler.h . . . 18
5.13 threads.h . . . 19
5.14 cache_control_mediator.h . . . 19
5.15 list.h . . . 20
5.16 log.h . . . 21
5.17 queue.h . . . 21

Предметный указатель 23

Глава 1

CacheControl — система управления страничным кэшем

CacheControl — это фоновый демон, который следит за заданными файлами или каталогами и управляет их присутствием в оперативной памяти (page cache). Демон может принудительно подгружать файлы в кэш для ускорения доступа (например, перед запуском тяжёлого приложения) или выгружать малоиспользуемые файлы, освобождая память.

1.1 Содержание

- Возможности
- Конфигурация
- Уровни логирования
- Использование
- Команды медиатора
- Сигналы
- Структура проекта
- Сборка
- Документация

1.2 Возможности

- Три политики кэширования:
  - keep — постоянно держать файл(ы) в кэше
  - evict — выгружать, если файл не использовался заданное время
  - monitor — только собирать статистику, не вмешиваясь
- Работа с масками — поддержка шаблонов в путях к файлам (/var/log/∗.log)
- Мониторинг страничного кэша — использование системного вызова mincore() для анализа присутствия страниц в памяти

- Отслеживание изменений — мониторинг файлов через inotify (события: изменение, удаление, перемещение)
- Управление через очередь сообщений — POSIX очередь для отправки команд демону
- Обработка сигналов — корректное завершение, перезагрузка конфигурации, отладочный вывод статистики
- Логирование — все события и ошибки направляются в syslog

1.2.1 Требования

- Компилятор GCC/Clang с поддержкой C11
- POSIX-совместимая ОС (Linux)

1.3 Конфигурация

1.3.1 Формат конфигурационного файла

<путь_с_маской> <политика> [параметры]

1.3.2 Поддерживаемые политики

Политика	Описание
keep	Постоянно держать файл(ы) в кэше (периодически подгружать при необходимости)
evict	Выгружать из кэша, если файл не использовался в течение заданного времени
monitor	Только собирать статистику, не вмешиваясь

1.3.3 Параметры

- age=<секунды> — время в секундах, после которого файл будет выгружен (для политики evict)
- interval=<секунды> — интервал сканирования (по умолчанию 60 секунд)

1.3.4 Пример конфигурационного файла

Файл /etc/cachecontrol.conf:

```
# База данных MySQL — всегда в кэше
/var/lib/mysql/* keep

# Логи выгружаются через час
/var/log/*.log evict age=3600

# Временные файлы выгружаются через 5 минут
/tmp/* evict age=300

# Файлы данных — только мониторинг
/opt/data/*.dat monitor

# Интервал сканирования
interval=120
```

1.4 Уровни логирования

Все сообщения демона записываются в syslog.

Уровень	Назначение
LOG_INFO	Информационные сообщения
LOG_WARNING	Предупреждения
LOG_ERR	Критические ошибки

1.5 Использование

1.5.1 Запуск демона

```
# Запуск с конфигом по умолчанию (/etc/cachecontrol.conf)
./bin/cache_control -d

# Запуск с указанием конфигурационного файла
./bin/cache_control -c /etc/cachecontrol.conf
```

1.5.2 Запуск медиатора

```
# Интерактивный режим управления демоном
./bin/cache_control_mediator
```

1.6 Команды медиатора

Команда	Аргумент	Описание
status	all	Вывести статистику по файлу или по всем отслеживаемым
cache	<file>	Принудительно подгрузить файл в кэш (игнорируя политику)
evict	<file>	Принудительно выгрузить файл из кэша
reload	—	Перечитать конфигурационный файл без остановки
exit	—	Корректно завершить работу демона
clear	—	Очистить терминал

1.7 Сигналы

Демон поддерживает обработку следующих сигналов:

Сигнал	Действие
SIGTERM / SIGINT	Корректное завершение работы (освобождение памяти, закрытие дескрипторов, удаление PID-файла)
SIGHUP	Перезагрузка конфигурационного файла без остановки демона
SIGUSR1	Вывод текущей статистики в syslog (для отладки)

Пример отправки сигналов:

```
# Перезагрузить конфигурацию
```

```
kill -SIGHUP <PID>

# Вывести статистику в syslog
kill -SIGUSR1 <PID>

# Завершить демон
kill <PID>
kill -SIGINT <PID>
```

1.8 Структура проекта

```
.
|-- bin/                        # Исполняемые файлы
|   |-- cache_control           # Демон
|   `-- cache_control_mediator  # Медиатор
|-- config/                     # Конфигурационные файлы
|   `-- cachecontrol.conf       # Пример конфига
|-- docs/                       # Документация (Doxygen)
|   |-- html/                   # Сгенерированная HTML-документация
|   `-- mainpage.md             # Главная страница документации
|-- include/                    # Заголовочные файлы (.h)
|   |-- cache_control/
|   |   |-- cache_control.h
|   |   |-- command_listener.h
|   |   |-- context.h
|   |   |-- daemonize.h
|   |   |-- events_stuff.h
|   |   |-- file_loading.h
|   |   |-- file_monitoring.h
|   |   |-- file_statistics.h
|   |   |-- inotify_stuff.h
|   |   |-- options.h
|   |   |-- parse_config.h
|   |   |-- signal_handler.h
|   |   `-- threads.h
|   |-- cache_control_mediator/
|   |   `-- cache_control_mediator.h
|   |-- list/
|   |   `-- list.h
|   |-- log/
|   |   `-- log.h
|   `-- queue/
|       `-- queue.h
|-- obj/                        # Объектные файлы (.o)
|   |-- cache_control/
|   |-- cache_control_mediator/
|   |-- list/
|   |-- log/
|   `-- queue/
|-- src/                        # Файлы с исходным кодом (.c)
|   |-- cache_control/
|   |   |-- command_listener.c
|   |   |-- context.c
|   |   |-- daemonize.c
|   |   |-- events_stuff.c
|   |   |-- file_loading.c
|   |   |-- file_monitoring.c
|   |   |-- file_statistics.c
|   |   |-- inotify_stuff.c
|   |   |-- main.c
|   |   |-- options.c
|   |   |-- parse_config.c
|   |   |-- signal_handler.c
|   |   `-- threads.c
|   |-- cache_control_mediator/
|   |   `-- main.c
|   |-- list/
|   |   `-- list.c
|   |-- log/
|   |   `-- log.c
|   `-- queue/
|       `-- queue.c
|-- Doxyfile
|-- Makefile
```

1.8.1 Модули демона (src/cache_control/)

Файл	Описание
main.c	Точка входа демона
parse_config.c	Парсер конфигурационного файла
context.c	Управление глобальным состоянием демона
daemonize.c	Запуск в фоновом режиме
signal_handler.c	Обработка сигналов
file_monitoring.c	Управление политиками кэширования
file_statistics.c	Сбор статистики через mincore()
file_loading.c	Управление страничным кэшем файлов
inotify_stuff.c	Мониторинг изменений файловой системы
command_↩ listener.c	Обработка команд медиатора
threads.c	Управление потоками демона
options.c	Обработка аргументов командной строки
events_stuff.c	Обработка событий файловой системы

1.8.2 Модули медиатора (src/cache_control_mediator/)

Файл	Описание
main.↩ c	Интерфейс управления демоном и отправка команд через очередь сообщений

1.9 Сборка

1.9.1 Доступные цели Make

```
make help          # Показать все доступные цели
make all           # Собрать все бинарные файлы
make clean         # Удалить объектные файлы
make clang_format  # Отформатировать исходный код с помощью clang-format
make docs          # Сгенерировать документацию
make docs_clean    # Удалить сгенерированную документацию
```

1.9.2 Сборка проекта

```
# Собрать демон и медиатор
make

# Собрать только демон
make cache_control

# Собрать только медиатор
make cache_control_mediator

# Удалить артефакты сборки
make clean
```

1.9.3 Генерация документации

```
# Сгенерировать HTML- и PDF-документацию
make docs

# Удалить сгенерированную документацию
make docs_clean
```

1.10 Документация

После выполнения
make docs

документация будет доступна:

- HTML: docs/html/index.html
- PDF: docs/latex/refman.pdf (если установлен LaTeX)

Проект документирован с использованием Doxygen. Документация включает:

- описание всех модулей проекта;
- описание структур данных;
- описание функций и их параметров;
- зависимости между модулями;

Глава 2

Алфавитный указатель структур данных

2.1 Структуры данных

Структуры данных с их кратким описанием.

cache_control 11
command_t 11
file 12
file_info 12
file_monitoring 12
file_stat 13
list 13
mediator_command 13
mqfd_t 13
node 14
replay_t 14
reply_t 14

Глава 3

Список файлов

3.1 Файлы

Полный список документированных файлов.

include/cache_control/cache_control.h . . . 15
include/cache_control/command_listener.h . . . 15
include/cache_control/context.h . . . 16
include/cache_control/daemonize.h . . . 16
include/cache_control/events_stuff.h . . . 16
include/cache_control/file_loading.h . . . 16
include/cache_control/file_monitoring.h . . . 17
include/cache_control/file_statistics.h . . . 17
include/cache_control/inotify_stuff.h . . . 18
include/cache_control/options.h . . . 18
include/cache_control/parse_config.h . . . 18
include/cache_control/signal_handler.h . . . 18
include/cache_control/threads.h . . . 19
include/cache_control_mediator/cache_control_mediator.h . . . 19
include/list/list.h . . . 20
include/log/log.h . . . 21
include/queue/queue.h . . . 21

Глава 4

Структуры данных

4.1 Структура cache_control

Поля данных

- int daemon_flag
- char config_path [PATH_SIZE]
- struct list * file_list
- int shutdown_flag
- int interval
- int reload_flag
- int stats_flag
- pthread_mutex_t mutex

Объявления и описания членов структуры находятся в файле:

- include/cache_control/context.h

4.2 Структура command_t

Поля данных

- char command [COMMAND_LEN]
- char path_to_file [PATH_LEN]

Объявления и описания членов структуры находятся в файле:

- include/cache_control_mediator/cache_control_mediator.h

4.3 Структура file

Поля данных

- char path [PATH_BUFF_SIZE]
- struct file_stat * stat
- int politic
- int age
- int wd
- int fd
- void * vmem_mapping

Объявления и описания членов структуры находятся в файле:

- include/list/list.h

4.4 Структура file_info

Поля данных

- char path [PATH_BUFF_SIZE]
- off_t size
- int number_of_pages
- int number_of_pages_cache
- int percent_cache
- struct timespec last_access
- int politic
- int age

Объявления и описания членов структур находятся в файлах:

- include/cache_control/command_listener.h
- include/cache_control_mediator/cache_control_mediator.h

4.5 Структура file_monitoring

Поля данных

- int epoll_fd
- int main_timer_fd
- int inotify_fd

Объявления и описания членов структуры находятся в файле:

- include/cache_control/file_monitoring.h

4.6 Структура file_stat

Поля данных

- off_t size
- int number_of_pages
- int number_of_pages_cache
- int percent_cache
- struct timespec last_access

Объявления и описания членов структуры находятся в файле:

- include/list/list.h

4.7 Структура list

Поля данных

- struct node * start
- int length
- int evict_files

Объявления и описания членов структуры находятся в файле:

- include/list/list.h

4.8 Структура mediator_command

Поля данных

- char com [COMMAND_LEN]
- char path_to_file [PATH_FILE]

Объявления и описания членов структуры находятся в файле:

- include/cache_control/command_listener.h

4.9 Структура mqfd_t

Поля данных

- mqd_t fds [2]

Объявления и описания членов структуры находятся в файле:

- include/cache_control_mediator/cache_control_mediator.h

4.10 Структура node

Поля данных

- struct file * nd_file
- struct node * nd_next

Объявления и описания членов структуры находятся в файле:

- include/list/list.h

4.11 Структура replay_t

Поля данных

- int status
- int last_msg
- struct file_info f

Объявления и описания членов структуры находятся в файле:

- include/cache_control/command_listener.h

4.12 Структура reply_t

Поля данных

- int status
- int last_msg
- struct file_info f

Объявления и описания членов структуры находятся в файле:

- include/cache_control_mediator/cache_control_mediator.h

Глава 5

Файлы

5.1 cache_control.h

5.2 command_listener.h

```
00001 #ifndef CACHECONTROL_LISTNER_H
00002 #define CACHECONTROL_LISTNER_H
00003
00004 #include "file_loading.h"
00005 #include "context.h"
00006 #include "../queue/queue.h"
00007 #include "../log/log.h"
00008 #include "file_statistics.h"
00009 #include "file_monitoring.h"
00010 #include "../list/list.h"
00011
00012 #define COMMAND_LEN 10
00013 #define PATH_FILE 255
00014
00015 struct file_info {
00016     char path[PATH_BUFF_SIZE];
00017
00018     off_t size;
00019     int number_of_pages;
00020     int number_of_pages_cache;
00021     int percent_cache;
00022     struct timespec last_access;
00023
00024     int politic;
00025     int age;
00026 };
00027 struct replay_t {
00028     int status;
00029     int last_msg;
00030     struct file_info f;
00031 };
00032
00033 struct mediator_command{
00034     char com[COMMAND_LEN];
00035     char path_to_file[PATH_FILE];
00036 };
00037
00038 int status_all(struct node* mynode, struct replay_t snd_msg, mqd_t snd_queue, struct cache_control *cc);
00039 int status_spec(struct file* myfile, struct replay_t snd_msg, mqd_t snd_queue, struct cache_control *cc);
00040 int do_cache(struct file* myfile, struct replay_t snd_msg, mqd_t snd_queue);
00041 int do_evict(struct file* myfile, struct replay_t snd_msg, mqd_t snd_queue, struct cache_control* cc);
00042 int do_reload(struct cache_control* cc, struct replay_t snd_msg, mqd_t snd_queue);
00043 int do_exit(struct cache_control* cc, struct replay_t snd_msg, mqd_t rcv_queue, mqd_t snd_queue);
00044 int command_listner(void* args);
00045
00046 #endif
```

5.3 context.h

```
00001 #pragma once
00002
00003 #ifndef CONTEXT_H
00004 #define CONTEXT_H
00005
00006 #include <pthread.h>
00007 #include "../list/list.h"
00008 #include "parse_config.h"
00009 #include "../log/log.h"
00010
00011 #define PATH_SIZE 255
00012
00013 struct cache_control {
00014     int daemon_flag;
00015     char config_path[PATH_SIZE];
00016     struct list *file_list;
00017     int shutdown_flag;
00018     int interval;
00019     int reload_flag;
00020     int stats_flag;
00021     pthread_mutex_t mutex;
00022 };
00023
00024 int init_context(struct cache_control *cc);
00025 int remove_context(struct cache_control *cc);
00026
00027 #endif
```

5.4 daemonize.h

```
00001 #pragma once
00002
00003 #ifndef DAEMONIZE_H
00004 #define DAEMONIZE_H
00005
00006 #include <stdio.h>
00007 #include <unistd.h>
00008 #include "../log/log.h"
00009
00010 int daemonize();
00011
00012 #endif
```

5.5 events_stuff.h

```
00001 #pragma once
00002
00003 #ifndef EVENTS_STUFF_H
00004 #define EVENTS_STUFF_H
00005
00006 #include <sys/timerfd.h>
00007 #include <sys/epoll.h>
00008 #include <unistd.h>
00009 #include <stdio.h>
00010 #include "../log/log.h"
00011
00012 #define TIMEOUT 1000
00013 #define MAX_NUM_OF_EVENTS 2
00014
00015 int set_timer(int timer_fd, time_t seconds);
00016 int add_fd_to_epoll(int epoll_fd, int fd);
00017
00018 #endif
```

5.6 file_loading.h

```
00001 #pragma once
00002
00003 #ifndef FILE_LOADING_H
00004 #define FILE_LOADING_H
00005
00006 #include <stdio.h>
00007 #include <unistd.h>
```

```
00008 #include <sys/mman.h>
00009 #include <errno.h>
00010 #include <fcntl.h>
00011 #include "../list/list.h"
00012 #include "../log/log.h"
00013
00014 #define READ_BUFF 4096
00015
00016 int map_file(struct file *myfile);
00017 int load_file(struct file *myfile);
00018 int unmap_file(struct file *myfile);
00019
00020 #endif
```

5.7 file_monitoring.h

```
00001 #pragma once
00002
00003 #ifndef FILE_MONITORING_H
00004 #define FILE_MONITORING_H
00005
00006 #include <stdio.h>
00007 #include <stdlib.h>
00008 #include <sys/epoll.h>
00009 #include <sys/timerfd.h>
00010 #include <unistd.h>
00011 #include <sys/inotify.h>
00012
00013 #include "context.h"
00014 #include "../list/list.h"
00015 #include "file_statistics.h"
00016 #include "file_loading.h"
00017 #include "events_stuff.h"
00018 #include "inotify_stuff.h"
00019
00020 #define INTERVAL 60 // заглушка!! Заменить после реализации парсинга
00021 #define MIN_PERCENT_CACHE 80
00022
00023 struct file_monitoring {
00024     int epoll_fd;
00025     int main_timer_fd;
00026     int inotify_fd;
00027 };
00028
00029 int init_file_monitoring(struct file_monitoring *fm);
00030 void *start_file_monitoring(void *args);
00031 int do_file_statistics(struct file *myfile);
00032 int do_file_loading(struct file *myfile);
00033 int map_all_files(struct list *list_files);
00034 int processing_all_files(struct list *list_files);
00035 int check_type_fd(int fd, struct file_monitoring *fm, struct list *file_list);
00036 int reload_config(struct cache_control *cc, struct file_monitoring *fm);
00037
00038 #endif
```

5.8 file_statistics.h

```
00001 #pragma once
00002
00003 #ifndef FILE_STATISTICS_H
00004 #define FILE_STATISTICS_H
00005
00006 #include <stdint.h>
00007 #include <stdio.h>
00008 #include <unistd.h>
00009 #include <sys/mman.h>
00010 #include <sys/stat.h>
00011 #include <errno.h>
00012 #include <fcntl.h>
00013 #include "../list/list.h"
00014 #include "../log/log.h"
00015
00016 off_t get_file_size(char *path);
00017 int get_number_of_pages(off_t file_size);
00018 int get_number_of_pages_cache(void *vmem_mapping, off_t size,
00019                               int number_of_pages);
00020 int get_percent_cache(int number_of_pages, int number_of_pages_cache);
00021 int get_last_access(char *path, struct timespec *last_access);
00022 int get_page_stat(struct file *myfile);
00023
00024 #endif
```

5.9 inotify_stuff.h

```
00001 #pragma once
00002
00003 #ifndef INOTIFY_STUFF_H
00004 #define INOTIFY_STUFF_H
00005
00006 #include <sys/inotify.h>
00007 #include <unistd.h>
00008 #include <string.h>
00009 #include <stdio.h>
00010 #include "../list/list.h"
00011 #include "file_loading.h"
00012 #include "file_statistics.h"
00013
00014 int add_all_files_to_inotify(int inotify_fd, struct list *file_list);
00015 int read_inotify(int inotify_fd, struct list *file_list);
00016 int remove_all_files_from_inotify(int inotify_fd, struct list *file_list);
00017 int file_in_attrib(int wd, struct list *file_list);
00018 int file_in_modify(int wd, struct list *file_list);
00019
00020 #endif
```

5.10 options.h

```
00001 #pragma once
00002 #ifndef OPTIONS_H
00003 #define OPTIONS_H
00004
00005 #include <stdio.h>
00006 #include <getopt.h>
00007 #include <string.h>
00008 #include <stdlib.h>
00009 #include <sys/stat.h>
00010 #include <limits.h>
00011 #include "../log/log.h"
00012 #include "context.h"
00013
00014 void print_info();
00015 int parse_options(int argc, char **argv, struct cache_control *cc);
00016 int validate_path_config(const char *path);
00017
00018 #endif
```

5.11 parse_config.h

```
00001 #ifndef PARSE_CONFIG_H
00002 #define PARSE_CONFIG_H
00003
00004 #include <stdio.h>
00005 #include <stdlib.h>
00006 #include <string.h>
00007 #include <fnmatch.h>
00008 #include <dirent.h>
00009 #include <limits.h>
00010 #include "../list/list.h"
00011
00012 #define DEFAULT_INTERVAL 60
00013 #define SIZE_BUFF 4096
00014
00015 int parse_config(const char *config_path, struct list *l, int *interval);
00016
00017 #endif
```

5.12 signal_handler.h

```
00001 #pragma once
00002
00003 #ifndef SIGNAL_HANDLER_H
00004 #define SIGNAL_HANDLER_H
00005
00006 #include <stdio.h>
00007 #include <signal.h>
00008 #include <stdlib.h>
00009 #include "context.h"
```

```
00010 #include "../log/log.h"
00011
00012 void sigint_handler(int sig);
00013 int register_handler(int signum, void (*handler)(int));
00014 void init_signal_handler();
00015 void set_context(struct cache_control *cc);
00016
00017 #endif
```

5.13 threads.h

```
00001 #pragma once
00002
00003 #ifndef THREAD_H
00004 #define THREAD_H
00005
00006 #include "context.h"
00007 #include <pthread.h>
00008 #include "command_listener.h"
00009 #include "file_monitoring.h"
00010
00011 int start_threads(struct cache_control *cc);
00012 int stop_threads();
00013
00014 #endif
```

5.14 cache_control_mediator.h

```
00001 #ifndef CACHE_CONTROL_MEDIATOR_H
00002 #define CACHE_CONTROL_MEDIATOR_H
00003
00004 #include <unistd.h>
00005 #include <string.h>
00006 #include <stdio.h>
00007 #include "../queue/queue.h"
00008 #include <sys/stat.h>
00009 #include "../list/list.h"
00010
00011 #define SEND_FILE "/snd_queue"
00012 #define RECV_FILE "/rcv_queue"
00013 #define CLEAR_TERMINAL "\033[2J\033[H"
00014 #define RED "\033[31m"
00015 #define DASHES "----------\n"
00016 #define N 255
00017 #define MAX_TOK 2
00018 #define PATH_LEN 255
00019 #define COMMAND_LEN 10
00020 #define PATH_BUFF_SIZE 255
00021 #define PATH_LOG_FILE "~/logs/cache_control_mediator.log"
00022 #define POLITIC_LEN 15
00023
00024 typedef struct {
00025     char command[COMMAND_LEN];
00026     char path_to_file[PATH_LEN];
00027 } command_t;
00028
00029 struct file_info {
00030     char path[PATH_BUFF_SIZE];
00031
00032     off_t size;
00033     int number_of_pages;
00034     int number_of_pages_cache;
00035     int percent_cache;
00036     struct timespec last_access;
00037
00038     int politic;
00039     int age;
00040 };
00041 typedef struct {
00042     int status;
00043     int last_msg;
00044     struct file_info f;
00045 } reply_t;
00046
00047 enum pol { P_KEEP = 1, P_EVICT, P_MONITOR };
00048
00049 typedef struct {
00050     mqd_t fds[2];
00051 } mqfd_t;
```

```
00052
00053 typedef enum {
00054     CMD_EXIT,
00055     CMD_RELOAD,
00056     CMD_EVICT,
00057     CMD_CACHE,
00058     CMD_STATUS,
00059     CMD_CLEAR,
00060     CMD_UNKNOWN
00061 } command_type_t;
00062
00063 int read_string(char *str);
00064 void clear_stdin();
00065 void print_menu();
00066 int get_command(char *str, command_t *command, command_type_t *op);
00067 int print_answer(const char *const answer, const command_t *const ptr_command,
00068          const command_type_t op);
00069 int split_tokens(char *str, char *tokens[], const char *delim,
00070          size_t *count_tok);
00071 mqfd_t open_queue(const char *send_path, const char *recv_path,
00072           const int send_flags, const int recv_flags);
00073 int set_command(const char *str, command_t *command, const char *path,
00074         const command_type_t op);
00075 command_type_t get_command_type(const char *cmd);
00076 int get_politic_str(reply_t *replay, char *str);
00077 #endif
```

5.15 list.h

```
00001 #pragma once
00002
00003 #ifndef LIST_H
00004 #define LIST_H
00005
00006 #include <malloc.h>
00007 #include <stdio.h>
00008 #include <string.h>
00009 #include <time.h>
00010 #include <unistd.h>
00011 #include <sys/mman.h>
00012 #include "../log/log.h"
00013
00014 #define PATH_BUFF_SIZE 255
00015
00016 enum politics { KEEP = 1, EVICT, MONITOR };
00017
00018 struct file_stat {
00019     off_t size;
00020     int number_of_pages;
00021     int number_of_pages_cache;
00022     int percent_cache;
00023     struct timespec last_access;
00024 };
00025
00026 struct file {
00027     char path[PATH_BUFF_SIZE];
00028     struct file_stat *stat;
00029     int politic;
00030     int age;
00031     int wd;
00032     int fd;
00033     void *vmem_mapping;
00034 };
00035
00036 struct node {
00037     struct file *nd_file;
00038     struct node *nd_next;
00039 };
00040
00041 struct list {
00042     struct node *start;
00043     int length;
00044     int evict_files;
00045 };
00046
00047 struct file_stat *init_file_stat(off_t s, int np, int npc, int pc,
00048                  struct timespec *la);
00049 struct file_stat *init_empty_stat();
00050 struct file *init_file(char *pb, int p, int a);
00051 int add_file_stat(struct file *f, struct file_stat *fs);
00052 struct node *init_node(struct file *f);
00053 struct list *init_list();
00054 int add_file(struct list *l, struct file *f);
00055 struct file *find_file_by_path(struct list *l, char *path);
```

```
00056 struct file *find_file_by_wd(struct list *l, int wd);
00057 int delete_file_by_path(struct list *l, char *path);
00058 int delete_file_by_wd(struct list *l, int wd);
00059 int free_list(struct list *l);
00060
00061 #endif
```

5.16 log.h

```
00001 #ifndef LOG_H
00002 #define LOG_H
00003 #include <stdio.h>
00004 #include <string.h>
00005 #include <syslog.h>
00006 #include <errno.h>
00007
00008 #define LOG_BUFF 512
00009
00010 void open_log();
00011 void logging(int log_level, char *msg);
00012 void close_log();
00013 #endif
```

5.17 queue.h

```
00001 #ifndef QUEUE_H
00002 #define QUEUE_H
00003 #include <malloc.h>
00004 #include <stdio.h>
00005 #include <stdlib.h>
00006 #include <string.h>
00007 #include <unistd.h>
00008 #include <sys/stat.h>
00009 #include <fcntl.h>
00010 #include <mqueue.h>
00011 #include <time.h>
00012 #include <errno.h>
00013
00014 #define SIZE_RCV 1024
00015 #define SIZE_SND 45
00016 #define MAX_SIZE_MSG 8192
00017 #define CHANGE_SIZE_MSG 8190
00018 #define MQ_TIMEOUT 5
00019
00020 mqd_t create_queue(char *file_desc, int flags, int size_msg);
00021 char *recive_msg(mqd_t queue, char *snd_msg, int size_msg);
00022 int send_msg(mqd_t queue, char *snd_msg, int size_msg);
00023 char *recive_msg_for_mediator(mqd_t queue_fd, char *rcv_msg, int size_msg);
00024
00025 #endif
```

Предметный указатель

cache_control, 11
CacheControl — система управления страничным кэшем, 1
command_t, 11

file, 12
file_info, 12
file_monitoring, 12
file_stat, 13

include/cache_control/cache_control.h, 15
include/cache_control/command_listener.h, 15
include/cache_control/context.h, 16
include/cache_control/daemonize.h, 16
include/cache_control/events_stuff.h, 16
include/cache_control/file_loading.h, 16
include/cache_control/file_monitoring.h, 17
include/cache_control/file_statistics.h, 17
include/cache_control/inotify_stuff.h, 18
include/cache_control/options.h, 18
include/cache_control/parse_config.h, 18
include/cache_control/signal_handler.h, 18
include/cache_control/threads.h, 19
include/cache_control_mediator/cache_control_mediator.h, 19
include/list/list.h, 20
include/log/log.h, 21
include/queue/queue.h, 21

list, 13

mediator_command, 13
mqfd_t, 13

node, 14

replay_t, 14
reply_t, 14